

KeenPay

Production Architecture & Systems Design

Internal architecture reference

Grow | Sell | Protect | Protocol Gateway

FastAPI | LangGraph | PostgreSQL | Redis | Next.js | Razorpay

Version 1.0.0 | August 2026

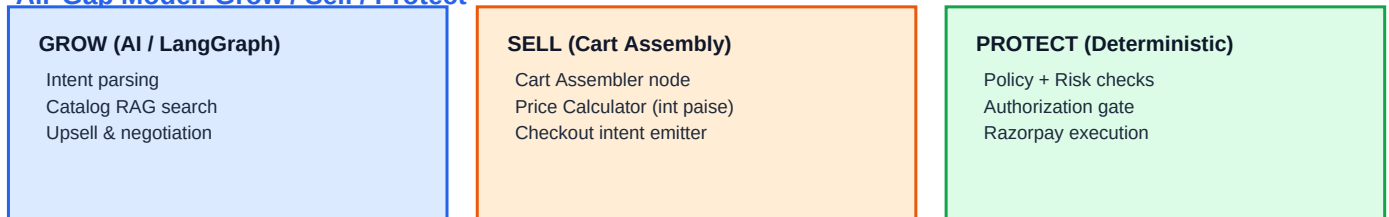
1. Engineering Philosophy

As a rule, AI models are non-deterministic and therefore implicitly untrusted with financial execution. KeenPay is designed with a strict Air-Gap philosophy: the AI engine handles the Grow (upselling, negotiation, discovery) and Sell (cart assembly) logic, but all financial API calls to Razorpay are firmly locked behind a deterministic Protect layer.

The V4 Protocol Gateway lets agentic-commerce protocols (MCP, ACP, UCP, AP2, A2A, x402, A2UI, NPCI UAP / UPI) enter through adapters - not through separate payment stacks. Every adapter normalizes to one KeenPay intent. One policy engine. One audit trail. No protocol bypasses the Control Plane.

We prioritize a pragmatic, monolithic-first API using FastAPI, PostgreSQL, and Redis. Simple enough to deploy quickly, robust enough for audit and concurrency.

Air-Gap Model: Grow / Sell / Protect

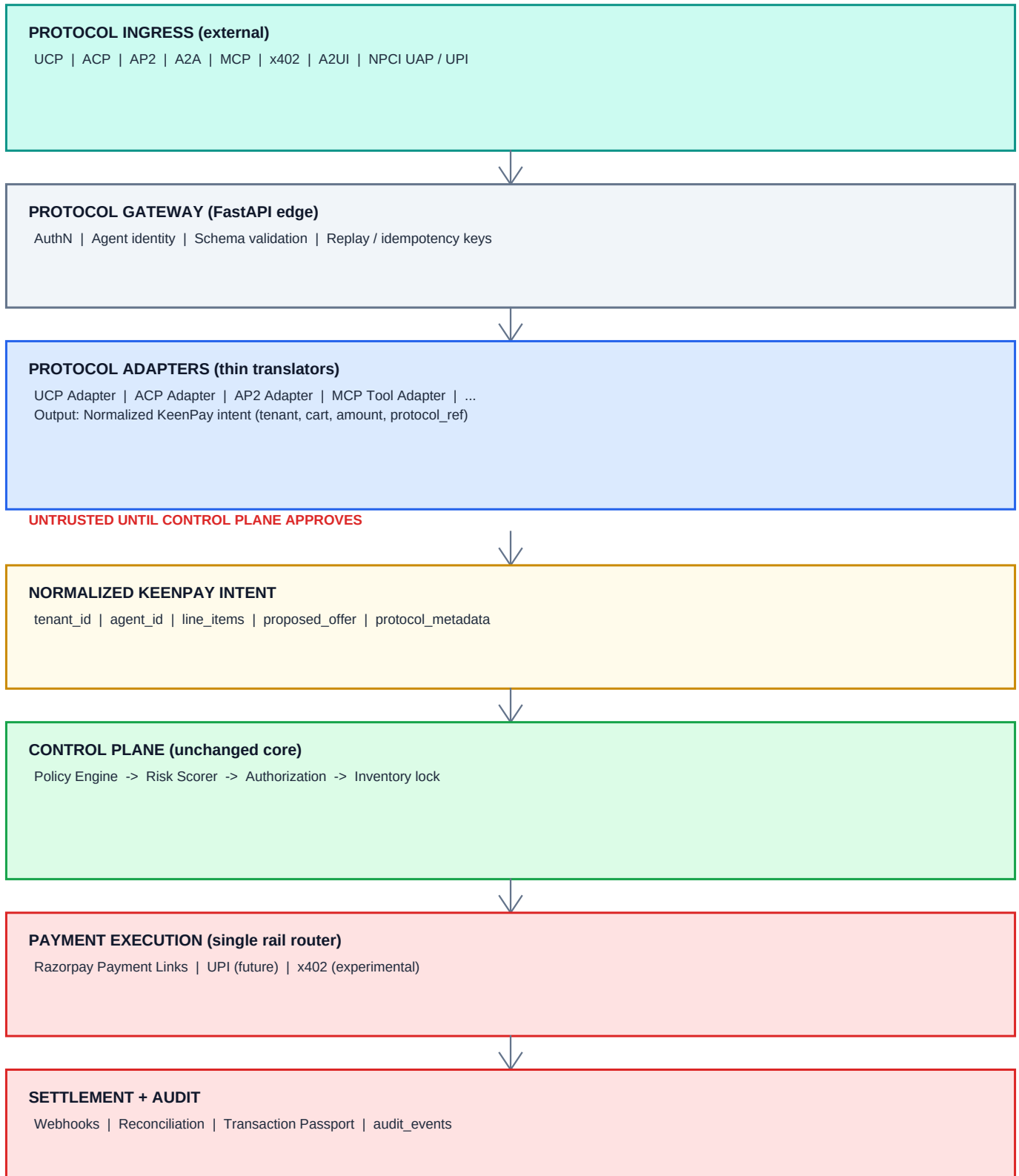


Trust Boundaries

- UNTRUSTED: User input, LLM output, inbound webhooks (validated on receipt)
- SEMI-TRUSTED: LangGraph orchestration (proposes actions, never pays)
- TRUSTED: Guardrail Engine, integer math, Razorpay client (gated side effects)

2. Protocol Gateway

Different protocols can enter KeenPay, but none can bypass the Control Plane. The Protocol Gateway is the single front door for agentic-commerce protocol traffic. It authenticates the caller, validates schema, maps protocol-specific messages to a normalized KeenPay intent, and forwards that intent to the existing graph and Control Plane. No adapter calls Razorpay, UPI, or x402 directly.



3. Protocol Catalog & Maturity

Status reflects architecture support today - not marketing claims. Core = in the live path. Adapter-ready = gateway contract defined, adapter can ship. Experimental = stub only. Future = planned rail, no adapter code yet.

Protocol	Role	Status	KeenPay mapping
MCP	Controlled agent tools & context	Core	Tool allowlist on agents table; MCP server maps to approved tools only
ACP	Agentic checkout / commerce	Core	Same as SELL path: intent -> guardrail -> pay
A2UI	Agent-driven UI components	Adapter-ready	Split UI: chat + trace; A2UI renders into hosted surfaces
UCP	Commerce interoperability	Adapter-ready	Adapter normalizes catalog/cart to KeenPay intent
AP2	Payment mandates & verifiable auth	Adapter-ready	Maps to scoped, expiring, single-use authorizations
A2A	Agent-to-agent messaging	Experimental	Gateway accepts A2A envelope; no production adapter yet
x402	Machine / pay-per-use HTTP payments	Experimental	Rail router stub; does not bypass Control Plane
NPCI UAP	India unified agentic payments	Future	Architecture slot in rail router; no live integration
UPI	India instant payments	Future	Settlement rail behind same authorization object

What we do NOT do

- Separate payment systems per protocol
- LLM-direct Razorpay / UPI / x402 calls
- Protocol-specific policy engines
- Claiming live UPI/x402 until adapter + rail tests pass

Gateway security (per adapter)

- Authentication: mTLS or signed JWT per protocol; no anonymous money ingress
- Agent identity: agents table - type, scopes, allowed_tools, trust_level, expiry
- Schema validation: Pydantic / JSON Schema per adapter; reject malformed payloads
- Replay protection: nonce + timestamp window; idempotency_keys per scope
- Scoped authorization: AP2 mandates bound to cart_hash + amount; single-use

4. Protocol Workflow

Every protocol, native chat, and external agent resolves to the same ordered lifecycle. The gateway does not shorten it; the protocol only chooses how it enters, never what it may do without approval.

Ordered lifecycle

1. Ingress

Raw envelope + auth, nonce, idempotency key

2. Gateway

AuthN agent identity / schema validation / replay protection

3. Intent

Normalized KeenPayIntent (tenant, agent, cart, protocol_ref)

4. GROW

parse intent, catalog search, negotiate offer (untrusted)

5. PROTECT

PolicyEngine -> RiskScorer -> Authorization -> Inventory lock

6. Gates

assert_payment_gates(); authorize within cart_hash + amount window

7. SELL

Reserve stock, orders(pending) + outbox, create payment link (idempotent)

8. Settle

Webhook HMAC + dedupe -> orders.paid -> audit_logs -> outbox publish

9. Fallback

REJECTED -> counter within policy; ESCALATED -> human ticket; UNKNOWN -> reconcile

Adapter contract

Each adapter is a pure function envelope -> KeenPayIntent. It performs no I/O side effects, holds no payment credentials, and never calls a rail. Amounts pass through only as an intent; the Control Plane recomputes totals in integer paise.

Protocol	Adapter entry	Gateway gate	Bound to
MCP	catalog.search / negotiate.offer	allowed_tools allowlist	agents.allowed_tools
ACP	checkout intent	same as SELL path	negotiation_sessions
AP2	mandate payload	issue authorizations (cart_hash + am	orders.authorization_id
A2UI	agent-rendered components	hosted surface render	UI only, no money
UCP	catalog/cart translate	schema normalize	KeenPayIntent
A2A	agent-to-agent envelope	ingress accept (experimental)	protocol_ref
x402	pay-per-use HTTP	rail stub, never bypasses CP	authorizations (experimental)
UPI / NPCI	settlement rail (future)	same authorization object	authorizations

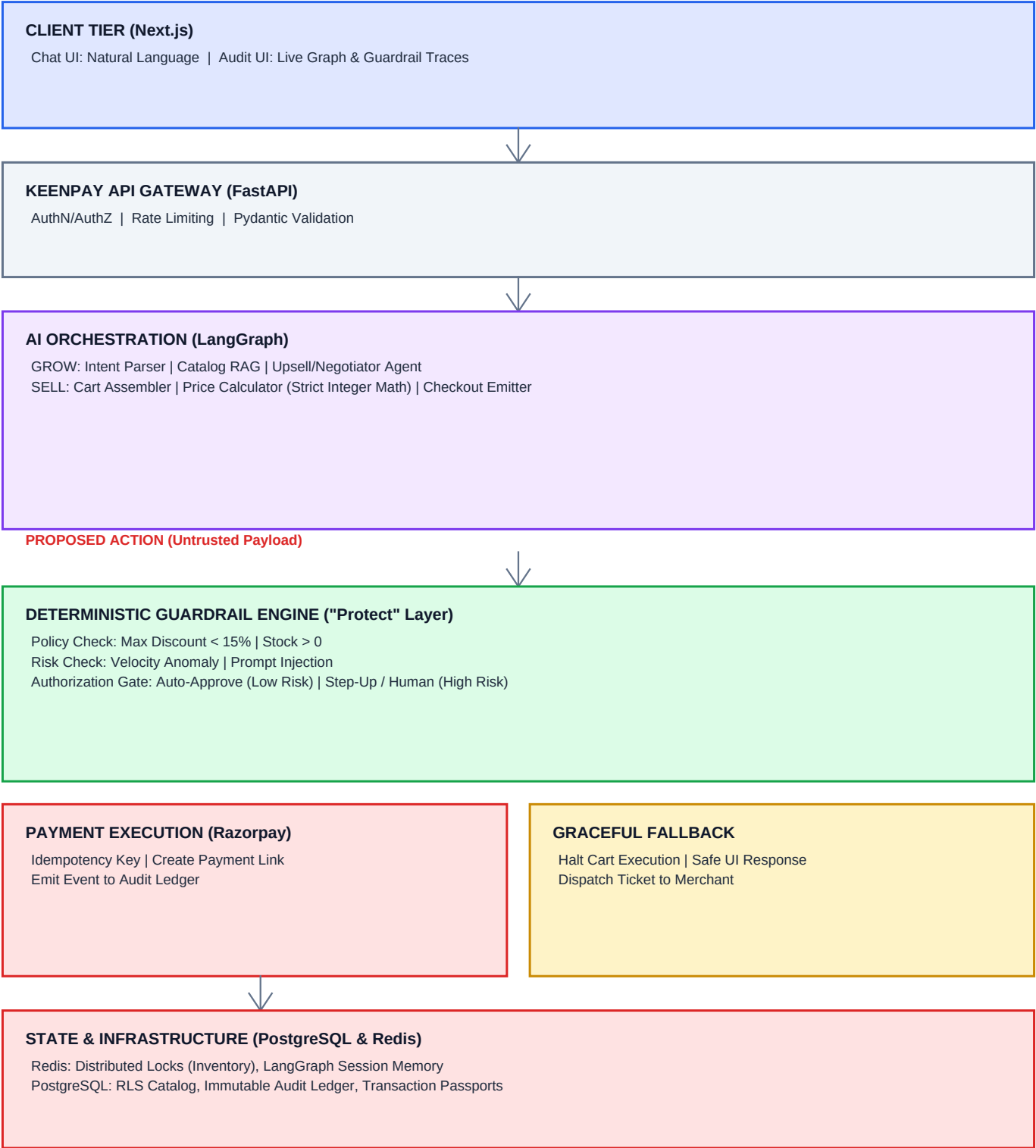
Authorization lifecycle (AP2 / scoped mandates)

A mandate is single-use and expiring. It is bound to a cart_hash and a [min, max] amount window so the amount cannot be swapped between issue and settle.

State	Transition
issued	consumed - charge bound to cart_hash, amount in window
issued	expired - TTL reached without a charge
issued	revoked - policy or user revokes
issued	denied - policy / risk rejects

5. System Architecture (Grow & Sell)

This architecture separates conversational intelligence from the financial control plane. The client tier uses HTTPS/WebSockets for chat and SSE for live audit traces.



6. Workflow Execution Steps

The lifecycle of a KeenPay interaction is strictly linear. An AI cannot skip a step.

Phase 1: GROW (Discovery & Revenue Optimization)

1. User Intent

Buyer asks: "I need a mechanical keyboard, but only if I can get a discount on two."

2. Context Retrieval

LangGraph queries PostgreSQL/pgvector catalog for mechanical keyboards and retrieves buyer LTV.

3. Agentic Upsell

Agent replies: "I can offer the Keychron K2. If you buy two, I can apply a 10% bundle discount."

4. User Agreement

User agrees. Cart Assembler constructs JSON: quantity=2, discount=10% (proposed, not final).

Phase 2: PROTECT (The Guardrail Interception)

Before any API call to Razorpay, the payload is intercepted by the Python Guardrail Engine.

1. Inventory Lock

Redis attempts to claim stock by 2. If it fails, checkout halts immediately.

2. Policy Assertion

Python asserts `proposed_discount <= merchant.max_discount_limit`. AI-hallucinated 50% is clamped to 15% or blocked.

3. Price Re-computation

Final total calculated in integer paise by backend, NOT the LLM, to prevent tampering.

4. Risk Scoring

Unusually large transactions trigger SEND TO HUMAN workflow, halting automated checkout.

Phase 3: SELL (Razorpay Execution & Settlement)

1. Idempotent Order Creation

Backend generates unique cart hash as Idempotency-Key. Calls Razorpay POST `/v1/payment_links`.

2. Link Delivery

Payment Link pushed via WebSocket to Next.js chat interface.

3. Webhook Reconciliation

User pays. Razorpay fires `payment_link.paid`. KeenPay verifies HMAC webhook signature.

4. Audit Commit

Final state written to `audit_ledger`. Transaction Passport links Razorpay Order ID to LLM node and policy version.

Linear Flow Enforcement

PROTOCOL -> GATEWAY -> INTENT -> GROW -> PROPOSE -> PROTECT -> [APPROVED] -> SELL ->
SETTLE | [REJECTED] -> FALLBACK

7. Hardcoded Guardrails (Security Matrix)

To guarantee system integrity, these guardrails are enforced at the API level. No LLM prompt can override them.

Guardrail Vector	Threat	KeenPay Deterministic Solution
Data Isolation	Cross-merchant data leaks	Row-Level Security (RLS) in PostgreSQL. Merchant A cannot query Merchant B catalog or orders.
Double Spend	Network timeout causes payment retr	Strict Idempotency + Never-Retry-Unknown protocol. Razorpay timeout marks state UNKNOWN; background worker polls.
Margin Erosion	Prompt injection forces Rs.1 offer	Absolute Price Floor Policy. Backend recalculates (Base Price * Qty) - Max Allowed Discount.
Concurrency	Two agents sell last item	Redis Distributed Locks claim inventory before Payment Link generation; release on timeout.
Untraceability	Cannot debug bad transaction	Transaction Passport: immutable log linking Razorpay Order ID to LLM prompt and policy version.
Prompt Injection	Bypass rules via chat	Deterministic regex + anomaly scorer; security_block halts money actions.

LLM Arithme

Webhook Ta

Authorization Gate Outcomes

AUTO-APPROVE	Low risk: discount within policy, stock available, anomaly score < 0.5
CLAMP	Discount exceeds cap: force to merchant.max_discount_limit
REJECT	Margin violation or invalid SKU: halt, explain to user
STEP-UP / HUMAN	High risk or max negotiation rounds: escalation ticket to merchant

8. Transaction Passport & Audit Ledger

Every finalized cart generates an immutable Transaction Passport stored in PostgreSQL audit_logs. This proves exactly which AI node proposed the discount and which policy version authorized it.

Passport Fields

<code>passport_id</code>	UUID v4 primary reference
<code>session_id</code>	LangGraph thread / negotiation session
<code>razorpay_order_id</code>	Razorpay Payment Link or Order ID
<code>offer_version</code>	Monotonic cart proposal version
<code>decision_id</code>	Guardrail evaluation reference
<code>policy_version</code>	e.g. 2026.08.1
<code>grow_node_trace</code>	JSONB: intent, catalog hits, negotiation rationale
<code>protect_node_trace</code>	JSONB: per-rule pass/fail/clamp results
<code>sell_node_trace</code>	JSONB: idempotency_key, link_id, webhook events
<code>final_amount_paise</code>	Deterministic integer total
<code>created_at</code>	UTC timestamp (append-only)

Observability Channels

- WebSocket trace:{session_id} - real-time graph node + guardrail events to Audit UI
- SSE /ws/v1/session - Server-Sent Events for live state transitions
- PostgreSQL audit_logs - durable append-only ledger (UPDATE/DELETE blocked by trigger)
- Redis pub/sub - fan-out trace events to connected clients

Deployment Stack

Frontend	Next.js 14 - Chat UI + Audit UI (split pane)
API	FastAPI - monolithic gateway + LangGraph runtime
Orchestration	LangGraph - KeenPayStateGraph with interrupt before payment
Database	PostgreSQL 15 - RLS, pgvector catalog, audit ledger
Cache	Redis 7 - locks, session memory, rate limits, pub/sub
Payments	Razorpay Payment Links API (test + production modes)